

Solving Recurrences in Algorithm Analysis

Understanding the foundation of algorithmic efficiency through mathematical induction, trees, and master theorem.



Substitution Method

Guess the solution and prove it by mathematical induction.



Recursion Tree

Visualize the recursion structure to sum up costs at each level.



Master Theorem

Apply a powerful formula for standard divide-and-conquer forms.

Example 1: Binary Search

Halving the Problem Space

Python

```
def binary_search(arr, target, left, right): if left > right: return -1 mid = (left + right) // 2 if arr[mid] == target: return mid elif arr[mid] > target: # Recurse on left half return binary_search(arr, target, left, mid - 1) else: # Recurse on right half return binary_search(arr, target, mid + 1, right)
```

RECURRENCE RELATION

$$T(n) = T(n/2) + O(1)$$

BASE CASE

$$T(1) = O(1)$$

Example 1: Solution

Substitution Method

STEP 1

Guess the Solution

We conjecture that $T(n) = O(\log n)$.

Specifically, we want to prove:

$$T(n) \leq c \log n$$

for some constant $c > 0$

STEP 2

Base Case & Hypothesis

Base Case: $T(1) = 1$. Since $\log 1 = 0$, we adjust the base case to $T(2)$ or assume $T(1) \leq c$ for $c \geq 1$.

Inductive Hypothesis:

Assume $T(k) \leq c \log k$ for all $k < n$.

STEP 3

Inductive Step

$$T(n) = T(n/2) + 1$$

$$\leq c \log(n/2) + 1$$

→

$$= c(\log n - \log 2) + 1$$

$$= c \log n - c + 1$$

→

$$\leq c \log n$$

(True if $c \geq 1$)

Since the inductive step holds for $c \geq 1$, we have proven:

$$T(n) = \Theta(\log n)$$

Example 2: Merge Sort

Classic Divide and Conquer

RECURRENCE RELATION

$$T(n) = 2T(n/2) + O(n)$$

BASE CASE

$$T(1) = O(1)$$

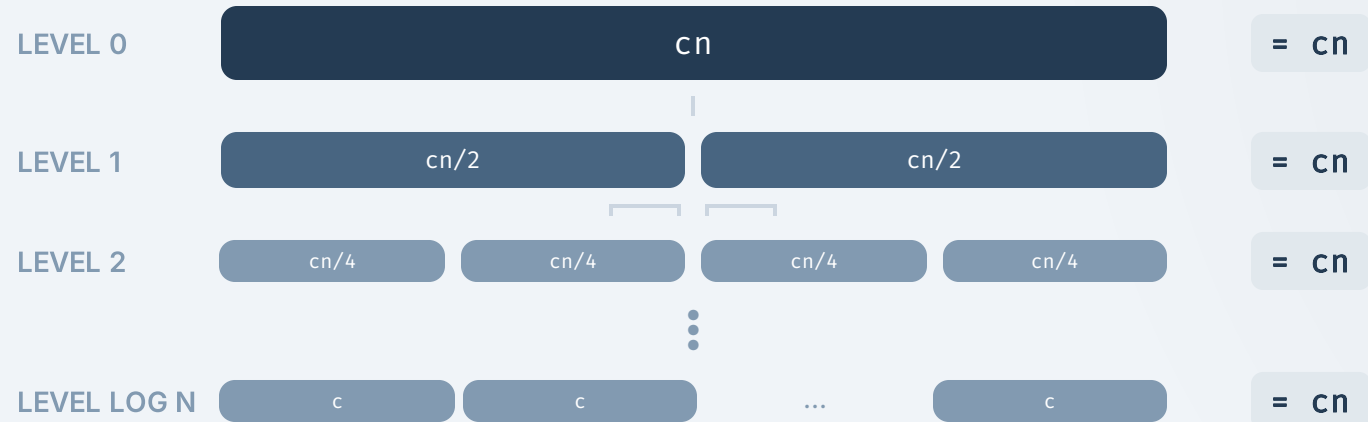
Python

```
def merge_sort(arr):  
    if len(arr) ≤ 1: return arr  
    mid = len(arr) // 2 # Two recursive calls  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    # O(n) merge operation  
    return merge(left, right)
```

Example 2: Solution



Recursion Tree Method



TREE HEIGHT

Since problem size divides by 2 at each step,
height = $\log_2 n$

COST PER LEVEL

Sum of costs at each level is constant: cn

TOTAL LEVELS

Levels 0 to $\log_2 n$ implies $\log_2 n + 1$ levels

Total Complexity

$$\theta(n \log n)$$

Example 3: Linear Search

Worst Case Analysis

RECURRENCE RELATION

$$T(n) = T(n-1) + O(1)$$

BASE CASE

$$T(1) = O(1)$$

Python

```
def linear_search(arr, target, index=0): # Base
    case: end of list if index ≥ len(arr): return -1 #
    Base case: found target if arr[index] == target:
    return index # Recursive step: check next index
    return linear_search(arr, target, index + 1)
```

Example 3: Solution

Substitution Method

STEP 1

Guess

We conjecture that $T(n) = O(n)$. Goal: Prove $T(n) \leq cn$ for some $c > 0$.

STEP 2

Base Case

For $n = 1$: $T(1) = 1$. Goal: $1 \leq c(1)$. **True for $c \geq 1$**

STEP 3

Inductive Step

Assume $T(k) \leq ck$ for all $k < n$.

$$\begin{aligned} T(n) &= T(n-1) + 1 \\ &\leq c(n-1) + 1 \quad (\text{by hypothesis}) \\ &= cn - c + 1 \\ &\leq cn \end{aligned}$$

This holds if: $-c + 1 \leq 0 \Rightarrow c \geq 1$

Since the base case and inductive step hold for $c \geq 1$:

$$T(n) = \Theta(n)$$

Example 4: Tower of Hanoi

 Exponential Growth

 Implementation

PYTHON

```
def hanoi(n, source, target, aux):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return 1

    # 1. Move n-1 disks to auxiliary
    moves = hanoi(n-1, source, aux, target)

    # 2. Move largest disk to target
    print(f"Move disk {n} ... ")
    moves += 1

    # 3. Move n-1 disks from auxiliary to target
    moves += hanoi(n-1, aux, target, source)

    return moves
```

RECURRENCE RELATION

$$T(n) = 2T(n-1) + 1$$

Two recursive calls of size $n-1$ plus one constant operation (moving the largest disk).

BASE CASE

$$T(1) = 1$$

Moving a single disk takes constant time $O(1)$.

→ Structure: $T(n) = 2T(n-1) + O(1)$

Example 4: Solution

Substitution Method

STEP 1: GUESS

We conjecture exponential growth:

$$T(n) \leq c \cdot 2^n - d$$

Using a subtraction term (-d) is helpful.

STEP 2: BASE CASE

For $n = 1$, $T(1) = 1$:

$$1 \leq c \cdot 2^1 - d$$

$$1 \leq 2c - d$$

True if we choose $c=1$, $d=1$.

STEP 3: INDUCTIVE STEP

Assume $T(k) \leq c \cdot 2^k - d$ for $k < n$.

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &\leq 2(c \cdot 2^{n-1} - d) + 1 \\ &= c \cdot 2^n - 2d + 1 \\ &\leq c \cdot 2^n - d \end{aligned}$$

This inequality holds if:

$$-2d + 1 \leq -d$$

$$1 \leq d$$

Since $d \geq 1$ satisfies the condition:

$$T(n) = \Theta(2^n)$$

Example 5: Fibonacci

⚠ Naive Recursive - Exponential Redundancy

```
def fibonacci(n): # Base cases if n ≤ 1:  
    return n # Two recursive calls return  
    fibonacci(n-1) + fibonacci(n-2)
```

Python

RECURRENCE RELATION

$$T(n) = T(n-1) + T(n-2) + O(1)$$

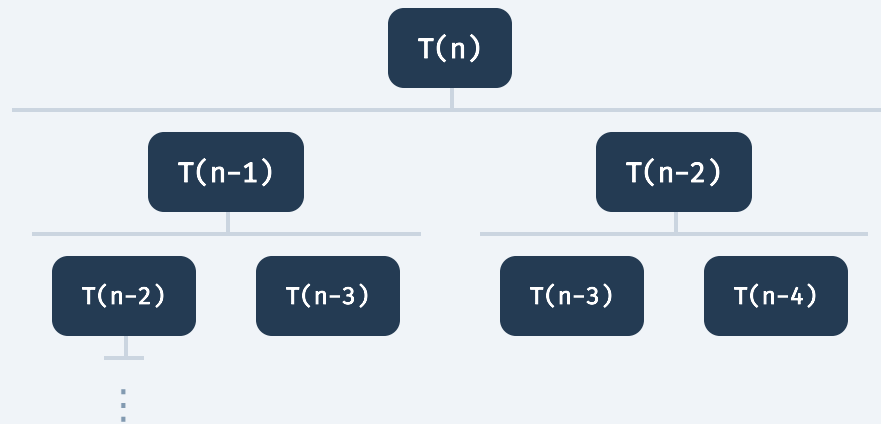
BASE CASES

$$T(0) = T(1) = O(1)$$

Observation: Unlike divide-and-conquer where subproblems are fractions of n (e.g., $n/2$), here subproblems decrease only by a constant (1 and 2).

Example 5: Solution

Recursion Tree Method



Tree is unbalanced and deep

TREE STRUCTURE

The tree is not balanced.

- **Max Depth:** n (leftmost path)
- **Min Depth:** $n/2$ (rightmost path)

NODE COUNT BOUNDS

Lower Bound: $\Omega(2^{n/2})$

Upper Bound: $O(2^n)$

Both are exponential.

EXACT SOLUTION (GOLDEN RATIO)

$$T(n) = \Theta(\varphi^n) \approx \Theta(1.618^n)$$

where $\varphi = (1 + \sqrt{5}) / 2$

Example 6: Karatsuba

⚡ Subquadratic Divide-and-Conquer

RECURRENCE RELATION

$$T(n) = 3T(n/2) + O(n)$$

Key Insight: Standard multiplication requires 4 sub-problems. Karatsuba reduces this to 3 by computing the middle term cleverly.

BASE CASE

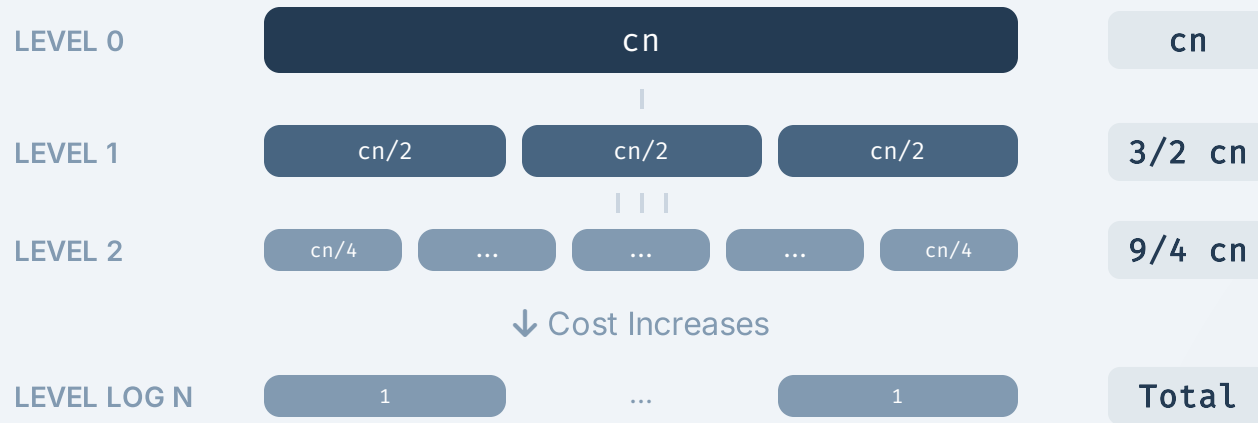
$$T(1) = O(1)$$

```
def karatsuba(x, y):  
    if x < 10 or y < 10: return x * y  
    n = max(len(str(x)), len(str(y))); m = n // 2  
    high1, low1 = divmod(x, 10**m); high2, low2 = divmod(y, 10**m)  
    z0 = karatsuba(low1, low2); z2 = karatsuba(high1, high2)  
    z1 = karatsuba(low1 + high1, low2 + high2) - z0 - z2  
    return z2 * 10**(2*m) + z1 * 10**m + z0
```

Python

Example 6: Solution

Recursion Tree Method



TREE PARAMETERS

Height = $\log_2 n$

Leaves = $3^{(\log_2 n)} = n^{(\log_2 3)}$

Cost at level $k = (3/2)^k cn$

GEOMETRIC SERIES SUM

Total = $cn \times \sum (3/2)^k$

Since ratio $r = 1.5 > 1$, the last term dominates.

Sum $\approx$ Last Term $\times$ Constant

Complexity determined by leaf level

$\theta(n^{1.585})$

(n to the power of $\log_2 3$)

Example 7: Strassen's Algorithm

Matrix Multiplication

RECURRENCE RELATION

$$T(n) = 7T(n/2) + O(n^2)$$

KEY DIFFERENCE

Standard multiplication uses **8** recursive calls.

Strassen reduces this to **7** calls using clever algebra, at the cost of more additions $O(n^2)$.

Python (Conceptual)

```
def strassen(A, B): n = len(A) if n == 1: return [[A[0][0] * B[0][0]]] # 1. Divide matrices into quadrants # (A11, A12, A21, A22, etc.) # 2. Compute 7 products (recursively) p1 = strassen(A11+A22, B11+B22) p2 = strassen(A21+A22, B11) # ... (p3 through p6) p7 = strassen(A12-A22, B21+B22) # 3. Combine results using additions c11 = p1 + p4 - p5 + p7 # ... calculate other quadrants return combine(c11, c12, c21, c22)
```

Example 7: Solution

Recursion Tree Method



TREE PARAMETERS

Height = $\log_2 n$

Leaves = $7^{(\log_2 n)} = n^{(\log_2 7)}$

Cost at level $k = (7/4)^k \text{ } cn^2$

GEOMETRIC SERIES SUM

Total = $cn^2 \times \sum (7/4)^k$

Ratio $r = 1.75 > 1$.

The cost is dominated by the leaf level.

Complexity determined by leaves

$\theta(n^{2.807})$

(Better than $O(n^3)$)

The Master Theorem

A Powerful Tool for Divide-and-Conquer Recurrences

$$T(n) = aT(n/b) + f(n)$$

$a \geq 1$: number of subproblems

$b > 1$: factor by which input size shrinks

$f(n)$: cost of dividing and combining

Compare $f(n)$ vs $n^{\log_b a}$

CASE 1

If $f(n)$ is polynomially **smaller** than the watershed function:

$$f(n) = O(n^{\log_b a - \epsilon})$$

SOLUTION

$$\Theta(n^{\log_b a})$$

CASE 2

If $f(n)$ is **equal** to the watershed function (possibly with log factors):

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

SOLUTION

$$\Theta(n^{\log_b a} \log^{k+1} n)$$

CASE 3

If $f(n)$ is polynomially **larger** (and satisfies regularity condition):

$$f(n) = \Omega(n^{\log_b a + \epsilon})$$

SOLUTION

$$\Theta(f(n))$$

Example 1: Binary Search

 Master Theorem Application

PARAMETERS

$$T(n) = T(n/2) + 1$$

a 1

b 2

f(n) 1

ANALYSIS STEPS

1 Calculate Watershed Function
 $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$

2 Compare f(n) with Watershed
 $f(n) = 1 = \Theta(1)$
They are asymptotically equal.

3 Identify Case
Case 2 applies (k=0)

$$T(n) = \Theta(n^0 \log n) =$$

$$\Theta(\log n)$$

Master Theorem Example 2

 Merge Sort Revisited

$$T(n) = 2T(n/2) + O(n)$$

1. IDENTIFY PARAMETERS

a (subproblems) 2

b (divisor) 2

f(n) (cost) n

2. COMPARE WITH WATERSHED

Calculate $n^{\log_b a}$:

$$n^{\log_2 2} = n^1 = n$$

Compare f(n) with n:

$$f(n) = \Theta(n)$$

CASE 2

Since f(n) matches the watershed (k=0):

$$\Theta(n \log n)$$

Example 3: Karatsuba

⚡ Master Theorem Case 1

$$T(n) = 3T(n/2) + O(n)$$

$$a = 3$$

$$b = 2$$

$$f(n) = n$$

WORK AT ROOT $F(N)$

$$n^1$$

<

WATERSHED $N^{\log_B A}$

$$n^{\log_2 3}$$

$$\approx n^{1.585}$$

DOMINATES

CASE 1 APPLIES

Since $f(n)$ is polynomially smaller than the watershed, the recursion tree leaves dominate the cost.

$$\Theta(n^{\log_2 3})$$

Example 4: Strassen's Algorithm

Master Theorem Case 1

$$T(n) = 7T(n/2) + \Theta(n^2)$$

1. PARAMETERS

Subproblems (a):

7

Divisor (b):

2

Cost $f(n)$:

n^2

2. WATERSHED EXPONENT

Calculate $\log_b a$:

$$\log_2 7 \approx 2.81$$

Since $2.81 > 2$, the recursive part dominates.

3. COMPARISON

$f(n)$ vs $n^{2.81}$

$$n^2 = O(n^{2.81 - \epsilon})$$

*$f(n)$ is polynomially smaller.
Case 1 applies.*

Solution: $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$

Example 5: Case 3

✓ Polynomial Dominance

$$T(n) = 2T(n/2) + n^2$$

1. WATERSHED

$$a = 2, b = 2$$

Calculate $n^{\log_b a}$:

$$n^{\log_2 2} = n^1$$

2. COMPARISON

$$f(n) = n^2$$

Compare n^2 vs n^1 :

$$n^2 = \Omega(n^{1+\varepsilon})$$

(with $\varepsilon = 1$)

$f(n)$ is larger

3. REGULARITY CONDITION

MUST VERIFY: $A \cdot F(N/B) \leq C \cdot F(N)$

$$2 \cdot (n/2)^2 \leq c \cdot n^2$$

$$2 \cdot (n^2/4) \leq c \cdot n^2$$

$$n^2/2 \leq c \cdot n^2$$

✓ True for $c = 1/2$

Since $f(n)$ dominates and is regular, the solution is $f(n)$:

$$\theta(n^2)$$

Example 6: Another Case 3

Cubic Overhead

1. SETUP & COMPARISON

$$T(n) = 4T(n/2) + n^3$$

Watershed Function:

$$n^{\log_2 4} = n^2$$

Comparison:

$$n^3 \text{ vs } n^2$$

$f(n)$ is polynomially larger ($\Omega(n^{2+\epsilon})$).

2. REGULARITY CHECK

Verify $a \cdot f(n/b) \leq c \cdot f(n)$:

$$\begin{aligned} & 4 \cdot (n/2)^3 \\ &= 4 \cdot (n^3/8) \\ &= n^3/2 \\ &= 0.5 n^3 \end{aligned}$$

✓ Holds for $c \geq 0.5$

Since $c < 1$ exists, the condition is satisfied.

Solution (dominated by root): $\Theta(n^3)$

Example 7: Median-of-Medians

⚠ Non-Standard Recurrence

$$T(n) = T(n/5) + T(7n/10) + O(n)$$

WHY MASTER THEOREM FAILS

The standard form requires a **single** subproblem size reduction factor **b**.

Here we have two different factors:

$$b_1 = 5$$

$$b_2 = 10/7 \approx 1.43$$

We cannot simply plug this into the formula.

ANALYSIS APPROACH

Check the sum of the relative costs:

$$1/5 + 7/10 = 2/10 + 7/10 = 9/10$$

Since $9/10 < 1$, the total work decreases geometrically at each level.

The root level dominates.

Solution (via Substitution):

$$T(n) = \Theta(n)$$

Example 8: Ternary Search

🔍 Divide into 3, Solve 1

$$T(n) = T(n/3) + 1$$

One recursive call on 1/3 of the data.
Constant work to decide direction.

PARAMETERS

a (subproblems)	1
-----------------	---

b (reduction factor)	3
----------------------	---

f(n) (overhead)	1
-----------------	---

ANALYSIS

Calculate Watershed:

$$n^{\log_3 1} = n^0 = 1$$

$$f(n) = \Theta(n^{\log_b a})$$

CASE 2 APPLIES

Example 9: Quadruple Recursion

 High Branching Factor

$$T(n) = 4T(n/2) + n$$

1. PARAMETERS

$$a = 4$$

$$b = 2$$

$$f(n) = n$$

2. WATERSHED

Calculate $n^{\log_b a}$:

$$n^{\log_2 4} = n^2$$

3. COMPARISON

Compare n vs n^2 :

$$n = O(n^{2-\epsilon})$$

$f(n)$ is polynomially smaller.
Case 1 applies.

Solution (dominated by leaves): $\Theta(n^2)$

Example 10: Logarithmic Overhead

 Extended Case 2

$$T(n) = 2T(n/2) + n \log n$$

1. WATERSHED

$$n^{\log_2 2} = n$$

Standard watershed function.

2. COMPARISON

$$n \log n \text{ vs } n$$

$f(n)$ is larger by a factor of $\log n$.

3. IDENTIFY K

$$k = 1$$

$f(n) = \Theta(n \cdot \log^1 n)$
Fits Extended Case 2.

Rule: If $f(n) = \Theta(n^{\log_b a} \log^k n)$, then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.

Example 11: Square Root Recursion

↔ Variable Substitution

$$T(n) = 2T(\sqrt{n}) + \log n$$

Problem: $\sqrt{n}$ is not n/b . Master Theorem does not apply directly.

1. SUBSTITUTE

Let $m = \log n$.

Then $n = 2^m$.

$$S(m) = 2S(m/2) + m$$



2. SOLVE S(M)

Apply Master Theorem:

$a=2, b=2, f(m)=m$.

Case 2 applies.

$$S(m) = \theta(m \log m)$$



3. REVERT

Substitute back:

$m \rightarrow \log n$

$$T(n) = \theta(\log n \cdot \log(\log n))$$

Example 12: When It Fails

⊘ Non-Polynomial Difference

$$T(n) = 2T(n/2) + n / \log n$$

COMPARISON

Watershed:

$$n^{\log_2 2} = n$$

Cost $f(n)$:

$$n / \log n$$

$f(n)$ is smaller than n , but is it **polynomially** smaller?

THE POLYNOMIAL GAP

$$\text{Ratio} = n / (n / \log n) = \log n$$

$\log n$ grows slower than n^ϵ for any $\epsilon > 0$.

The difference is not polynomial.
We fall into the gap between Case 1 and Case 2.

Conclusion: **Master Theorem Does Not Apply** (Use Recursion Tree Method)

Comparison of Methods

 Choosing the Right Tool

 SUBSTITUTION <i>"The Scalpel"</i> ✓ Most general method; works on almost any recurrence. ✓ Provides rigorous mathematical proof. ✗ Requires a good initial guess. ✗ Algebra can be tedious and error-prone.	 RECURSION TREE <i>"The Sketchpad"</i> ✓ Excellent for visualizing the flow of work. ✓ Best way to generate a guess for substitution. ✗ Summations can get messy quickly. ✗ Less formal than substitution.	 MASTER THEOREM <i>"The Sledgehammer"</i> ✓ Fastest method; "cookbook" solution. ✓ Ideal for standard divide-and-conquer. ✗ Limited applicability (specific form only). ✗ Cannot handle non-constant gaps or complex $f(n)$.
--	---	---

RECOMMENDED WORKFLOW

Master Theorem → Recursion Tree → Substitution

Key Takeaways

★ Mastering Recurrence Analysis



THE HIERARCHY

Always start with the **Master Theorem** for speed.
If it fails, use the **Recursion Tree** to visualize.
Finally, use **Substitution** to prove your guess formally.



THE WATERSHED

The battle is always between the **Root Cost** $f(n)$ and the **Leaf Cost** $n^{\log_b a}$.
Whichever is polynomially larger determines the complexity.



COMMON PITFALLS

1. Don't forget the **Regularity Condition** in Case 3.
2. Watch out for the **Polynomial Gap** (e.g., n vs $n/\log n$).
3. Ensure a and b are constants.



THE GOLDEN RULE

Intuition + Rigor = Success.
Use trees to build intuition, but rely on math for the final answer.
Asymptotic notation (Θ , O , Ω) ignores constants, but constants matter in practice!